

Módulo 13: Patrones de diseño

Contenidos clave

Los patrones de diseño son soluciones que se repiten para problemas comunes de programación.

Creacionales: *Factory, Abstract Factory, Builder, Singleton* (cuándo evitarlo): Se enfocan en cómo crear objetos.

- **Factory:** decide qué objeto crear.
- **Abstract Factory:** crea familias de objetos relacionados.
- **Builder:** para construir objetos complejos paso a paso.
- **Singleton:** busca que exista una única instancia de una clase. No siempre es buena idea, puede generar estado global, dificultad para hacer pruebas, dependencias ocultas, problemas con concurrencia, debe usarse con cuidado.

Estructurales: *Adapter, Facade, Composite, Decorator, Proxy*: Se enfocan en cómo organizar y conectar objetos.

- **Adapter:** sirve para conectar dos interfaces incompatibles. Traduce una interfaz a otra.
- **Facade:** proporciona una interfaz sencilla para algo complejo. Simplifica un sistema complejo.
- **Composite:** permite tratar objetos individuales y grupos de objetos de la misma manera. Estructuras de árbol.
- **Decorator:** permite agregar comportamiento a un objeto sin modificar su código original.
- **Proxy:** actúa como intermediario antes de llegar al objeto real.

Comportamiento: *Strategy, Observer, Command, Mediator, Template Method, State*: Se enfocan en cómo interactúan los objetos y cómo se distribuyen las responsabilidades.

- **Strategy:** permite cambiar un algoritmo o comportamiento sin modificar el código que lo utiliza.
- **Observer:** permite que un objeto avise automáticamente a otros cuando ocurre algo.
- **Command:** convierte una acción en un objeto.
- **Mediator:** centraliza la comunicación entre varios objetos. Los componentes se comunican a través del mediador.
- **Template Method:** define una estructura general de un proceso, pero permite que algunos pasos sean diferentes.
- **State:** permite que un objeto cambie su comportamiento dependiendo de su estado.

Patrones idiomáticos: *decoradores, context managers, dataclasses*: Son interesantes porque Python ya tiene herramientas que funcionan naturalmente como patrones.

- **Decoradores:** permiten agregar comportamiento.
- **Context Managers:** garantizan que ciertos recursos se preparen y liberen correctamente.
- **Dataclasses:** simplifican la creación de clases principalmente destinadas a almacenar datos.

Objetivos

- Implementar patrones relevantes con ejemplos reales.

Se implementaron 3 patrones aplicados a un dominio de precios:

- Strategy: permitió intercambiar las reglas de cálculo mediante:

```
14 class RegularPricing: 22 class PercentageDiscountPricing:
38 class PremiumPricing:
```

- Decorator: se implementó con:

```
23 class CachedPricingStrategy:
```

 y el decorador funcional:

```
8 def cached price[
```

para agregar caché sin modificar las estrategias originales.

- Identificar antipatrones y señales de refactor.

Se evitó:

- Incluir múltiples reglas de precio dentro de una sola función condicional.
- Acoplar el servicio directamente a un proveedor externo.
- Modificar las estrategias originales para agregar caché.
- Duplicar lógica de acceso al catálogo.

La separación de responsabilidades mostró cuándo refactorizar: cuando una clase acumula reglas, depende directamente de infraestructura o necesita modificarse constantemente para agregar variantes.

Laboratorio

- Strategy para precios.

Se definió el protocolo PricingStrategy y varias implementaciones intercambiables:

```
14 class RegularPricing: 22 class PercentageDiscountPricing:
38 class PremiumPricing:
```

PricingService recibe cualquier estrategia compatible y calcula el precio sin conocer su implementación concreta.

- Decorator de caché.

Se implementó:

```
23 class CachedPricingStrategy:
8 def cached price[
```

La caché evita recalcular el precio para el mismo producto. Las pruebas verificaron que la estrategia original se ejecutará una sola vez.

- Adapter para proveedor externo.

Se creó:

```
26 class ProductCatalogAdapter:
```

El cuál transforma o traduce a externos cómo:

```
{  
    "display_name": "Keyboard",  
    "price": "100.00",  
}
```

Al modelo interno:

```
Product(  
    name="Keyboard",  
    base_price=Decimal("100.00"),  
)
```

- Pruebas con pytest

Se probaron:

- Cálculo de precio normal.
- Descuentos porcentuales.
- Precios premium.
- Uso correcto de la caché.
- Conversión del proveedor externo.
- Manejo de productos desconocidos.

Las pruebas se ejecutaron con pytest:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
poetry run pytest .\labs\day13_design_patterns\tests `
>> --cov=labs.day13_design_patterns `
>> --cov-report=term-missing `
>> --cov-report=html `
>> --cov-fail-under=80
===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Quiryat\Documents\Python\python-mastery-capstone
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default
_test_loop_scope=function
collected 7 items

labs\day13_design_patterns\tests\test_patterns.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.14.6-final-0
-----
Name                               Stmts  Miss  Cover   Missing
-----
labs\day13_design_patterns\adapters.py      15      0  100%
labs\day13_design_patterns\decorators.py     20      0  100%
labs\day13_design_patterns\domain.py         4      0  100%
labs\day13_design_patterns\services.py       7      0  100%
labs\day13_design_patterns\strategies.py    16      0  100%
-----
TOTAL                                     62      0  100%
Coverage HTML written to dir htmlcov
Required test coverage of 80% reached. Total coverage: 100.00%
===== 7 passed in 0.38s =====
```